BFOR516 - Supervised Learning

Srishti Gupta, Ph.D.

Machine Learning

1. **Supervised Learning**:
   a. **Definition**: In supervised learning, the model is trained on labeled data, meaning the input data is paired with the correct output.
   b. **Objective**: The goal is to learn a mapping from inputs to outputs, so that the model can predict the output for new, unseen inputs.
   c. **Examples**:
      i. Classification: Predicting a category (e.g., spam vs. not spam).
      ii. Regression: Predicting a continuous value (e.g., house prices).
   d. **Algorithms**: Linear Regression, Logistic Regression, Support Vector Machines (SVM), Neural Networks, Decision Trees, etc.

Machine Learning

1. **Unsupervised Learning:**
   a. **Definition**: In unsupervised learning, the model is trained on data without labeled responses. The system tries to learn the patterns and structure from the data.
   b. **Objective**: The goal is to find hidden patterns or intrinsic structures in the input data.
   c. **Examples:**
      i. Clustering: Grouping similar data points together (e.g., customer segmentation).
      ii. Dimensionality Reduction: Reducing the number of random variables to consider (e.g., PCA, t-SNE).
   d. **Algorithms:** K-Means Clustering, Hierarchical Clustering, Principal Component Analysis (PCA), etc.

Logistic Regression

Logistic Regression Introduction

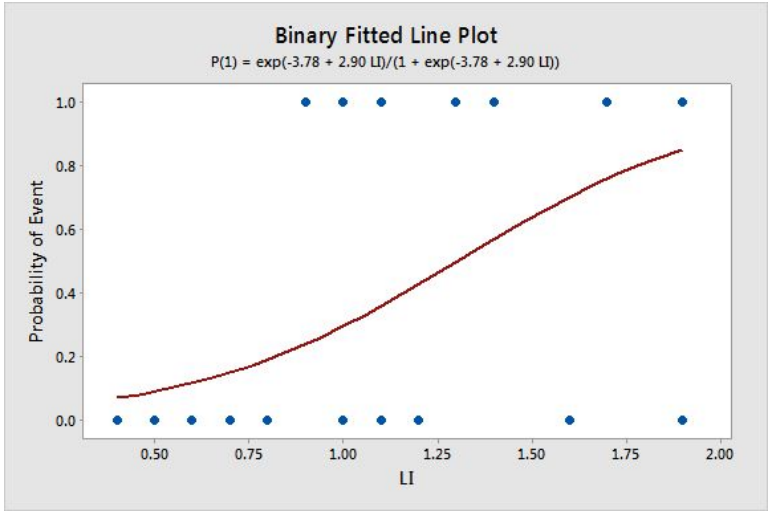
- Method used in statistics and data science to help predict a "yes or no" answer based on other information.
- Classification, supervised machine learning
- Example: Imagine a factory that makes parts. We could use logistic regression to find out if a part will crack (yes or no) based on certain measurements, like its size or chemical makeup.
- Probability: 0 to 1

Types of Logistic Regression

1. **Binary Logistic Regression**: This is used when there are only two possible outcomes. For example:
   a. Will the part crack or not? (yes or no)
   b. Will a student pass or fail a test? (pass or fail)
2. **Nominal Logistic Regression**: This is for situations where there are more than two outcomes, but they don't have any order. For example:
   a. What department does an employee work in? (e.g., marketing, sales, HR)
   b. What color is a product? (e.g., red, blue, green)
3. **Ordinal Logistic Regression**: This is used when there are more than two outcomes, and they have a natural order. For example:
   a. How effective is a course? (e.g., good, medium, poor)
   b. What's the condition of a patient? (e.g., stable, serious, critical)

Multiple Binary Logistic Regression

- Model predicts the probability of a "yes or no" outcome (called the success probability, π) based on several predictor variables.
- “S" shape, called a sigmoidal curve. The probability starts low (near 0), rises in the middle, and then flattens out near 1 as the predictor increases. This shape represents the transition from a "no" outcome to a "yes" outcome.



Likelihood Function

- Likelihood function calculates how likely the observed data is, given the values of the predictor variables. The goal is to find the βs (coefficients) that make the observed data most likely.
- **L(β; y, X)** is the likelihood function for binary logistic regression. It looks at all the observations (data points) and calculates the likelihood of the outcomes (e.g. like whether remission occurred in leukemia patients) based on the predictors (like the cellularity of the marrow, percentage of leukemia cells, etc.).
  - The log likelihood is just the natural logarithm of the likelihood function, which helps us simplify the calculations.

Example

Let's consider an example with leukemia patients:

- Response variable (Y): Whether the patient achieved remission (1 for yes, 0 for no).
- Predictor variables (X): Characteristics like:
  - Cellularity of the marrow clot (CELL)
  - Smear differential percentage of blasts (SMEAR)
  - Percentage of marrow leukemia cell infiltrate (INFIL)
  - Number of blasts in peripheral blood (BLAST)
  - Highest temperature before treatment (TEMP)
- We use logistic regression to predict the probability that a patient will go into remission based on these variables.
- The output of the logistic regression gives us coefficients (Coef) for each variable, which represent the effect of each predictor on the likelihood of remission.
  - For example, the coefficient for CELL is 30.8, but we also need to look at the p-value (0.554), which tells us how significant this predictor is. A high p-value suggests that CELL may not be a strong predictor of remission in this case.

Wald Test

- The Wald test is used to see if each predictor in a logistic regression model is important (i.e., if it significantly affects the outcome). In simple terms:
  - Each predictor has a coefficient (β\betaβ) in the logistic model, representing its effect on the outcome.
  - The Wald test checks if this coefficient is different from zero. If the coefficient is zero, the predictor has no effect.
  - The Wald test uses a ratio (essentially a "test statistic") that compares the size of the coefficient to its standard error (SE).
  - This ratio follows a standard normal distribution, meaning we can look up a p-value to determine if the predictor's effect is significant (like a t-test in linear regression).

Steps for Logistic Regression

1. Data preparation and feature engineering
2. Model training:
   a. select features
   b. split data into test and train datasets
   c. use LogisticRegression function from scikit-learn
   d. this model learns the relationship between the features and the target variable
3. Making predictions:
   a. We train the model using the training data. This tells the model how much weight (importance) to assign to each feature when predicting fraud. Once trained, we use the model to predict if new transactions (from the testing set) are fraudulent based on their features.
4. Model evaluation:
   a. Model Coefficients: We can view the model's coefficients (weights) for each feature. These coefficients tell us how much each feature influences the prediction of fraud. A positive coefficient indicates the feature increases the likelihood of fraud, and a negative coefficient suggests it decreases the likelihood.
   b. Confusion Matrix & Classification Report: These reports help us understand how well the model performs on the unseen testing data. They show how many transactions were correctly classified as fraudulent or non-fraudulent (true positives, true negatives, false positives, and false negatives).

Classification Report - Model Evaluation

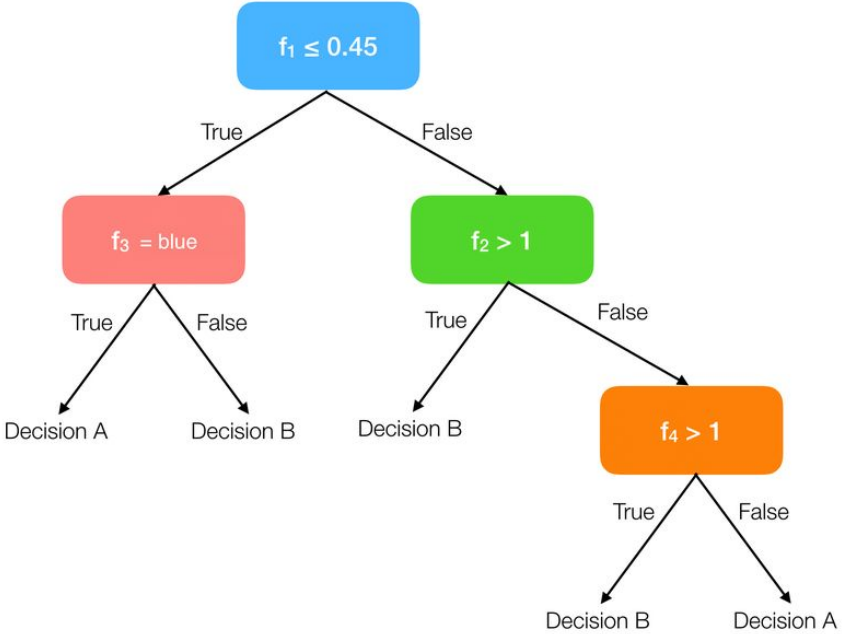
Performance evaluation tool used in machine learning to assess the effectiveness of a classification model.

1. **Precision**: The ratio of correctly predicted positive observations to the total predicted positives. It indicates how many of the predicted positive instances were actually positive.
   a. Good value: Precision close to 1 (or 100%)
2. **Recall (Sensitivity)**: The ratio of correctly predicted positive observations to all the actual positives. It shows how many of the actual positive instances were correctly identified.
   a. Good value: Recall close to 1
3. **F1-Score:** The harmonic mean of precision and recall. It balances both precision and recall, providing a single score that reflects both metrics.
   a. Good value: An F1-score close to 1
4. **Support:** The number of actual occurrences of the class in the dataset. This tells you how many instances of each class are present in the dataset.
   a. Good value: The support indicates the number of true instances for each class. A higher support means that the model is evaluated on a larger sample of that class, which often provides more reliable metrics.
5. **Accuracy (Overall)**: Accuracy close to 1 means the model is correctly predicting a high percentage of instances across all classes. However, accuracy alone can be misleading, especially in imbalanced datasets.

Decision Trees

Decision Trees

- A classification decision tree is a simple model used to make decisions or predictions.



Decision Trees

- A decision tree makes a statement and asks a question: Is this true or false?
- Based on the answer, the tree moves to the next step.
- When a decision tree classifies things into categories, it's called a classification tree.
- When it predicts numeric values, it's called a regression tree.

How do we decide the root node?

- The decision tree automatically selects the best question to start with by looking at the data and finding the feature (column of data) that gives the most information.
  - Information gain or Gini impurity to decide this.

How do we decide the root node?

- Information gain: This measure shows how well a question splits the data into pure groups. A "pure" group is one where all the data points fall into the same category. The higher the information gain, the better the question.
- Gini impurity: This calculates how "mixed" or "impure" the data is after a split. Lower impurity is better, meaning that after asking a question, the groups are mostly one class.
- The tree tries to minimize impurity.
- The decision tree goes through all possible questions (features) and selects the one that results in the best split of the data.

Bias-Variance Trade-off

- Shallow trees (with few splits) may underfit the data, leading to high bias (simplistic model that doesn't capture patterns).
- Deep trees (with many splits) can overfit, leading to high variance (model that's too sensitive to specific training data).
- The goal is to find a balance between bias and variance to get the best results.

Naïve Bayes Classifier

Naive Bayes Classifier

- The Naive Bayes classifier is a probabilistic machine learning model used for classification tasks.
- It is based on Bayes' Theorem and assumes that the features (predictors) are conditionally independent given the class label.
- This "naive" assumption simplifies the computation, making the algorithm fast and efficient, even for large datasets.

Key Concepts of Naive Bayes

1. **Bayes' Theorem**: Bayes' Theorem describes the probability of an event based on prior knowledge of conditions related to the event. It is mathematically expressed as:

   The formula is:

$$P(y|X) = \frac{P(X|y) \cdot P(y)}{P(X)}$$

   Where:

   - P(y|X) is the posterior probability, which tells us how likely a class y is, given the features X (like the words in the email).
   - P(X|y) is the likelihood, which tells us how likely we would see the features X (like certain words) given the class y (like "spam").
   - P(y) is the prior probability, which is the general likelihood of class y happening, before considering the features X.
   - P(X) is the marginal probability of the features X, which normalizes the result, ensuring the probabilities add up to 1.

Key Concepts of Naive Bayes

2. **Naive Assumption**: The Naive Bayes classifier assumes that all features are conditionally independent of each other given the class label. Mathematically, this looks like:

$$P(X|y) = P(x_1|y) \cdot P(x_2|y) \cdot \ldots \cdot P(x_n|y)$$

   a. Where each x_i represents a feature (like a word in the email).

Key Concepts of Naive Bayes

3. **Prediction**: For a given set of features X, the Naive Bayes classifier predicts the class y with the highest posterior probability:
   a. In simple terms, Naive Bayes predicts the class that makes the most sense, given the evidence (features) from the data:

$$\hat{y} = \arg\max_{y} P(y|X)$$

Key Concepts of Naive Bayes

To sum it up:

1. **Bayes' Theorem helps calculate probabilities based on known information.**
2. **Naive Assumption simplifies things by assuming features don't influence each other.**
3. **Prediction means choosing the class with the highest probability based on the data.**

Types of Naive Bayes Classifiers

1. **Gaussian Naive Bayes:**
   a. Assumes that continuous features follow a Gaussian (normal) distribution.
   b. Suitable for continuous data.
   c. Example: Classifying based on height, weight, etc.
2. **Multinomial Naive Bayes:**
   a. Used for discrete data, such as word counts in text classification.
   b. Assumes features represent the frequency of events (e.g., word occurrences).
   c. Example: Spam detection, document classification.
3. **Bernoulli Naive Bayes:**
   a. Used for binary/boolean features (e.g., presence or absence of a word).
   b. Assumes features are binary-valued.
   c. Example: Text classification where word presence is more important than frequency.

Multinomial Naive Bayes

Overview

- The Multinomial Naive Bayes classifier is a specific type of Naive Bayes algorithm designed for discrete data, particularly for problems where the features represent counts or frequencies of events.
- It is commonly used in text classification tasks, such as spam detection, sentiment analysis, and document categorization, where the features are often word counts or term frequencies.

Steps to Build a Multinomial Naive Bayes Classifier

1. **Data Preparation:**
   a. Preprocess the data (e.g., handle missing values, encode categorical variables).
   b. Split the data into training and testing sets.
2. **Calculate Priors:**
   a. Compute the prior probabilities $P(y)$ for each class from the training data.
3. **Calculate Likelihoods:**
   a. For each feature, compute the likelihood $P(x_i | y)$ based on the training data.
   b. For Gaussian Naive Bayes, calculate the mean and variance of each feature for each class.
4. **Make Predictions:**
   a. For a new input X compute the posterior probability $P(y | X)$ for each class using Bayes' Theorem.
   b. Predict the class with the highest posterior probability.

Example of Multinomial Naive Bayes Classifier

Imagine you want to classify emails you receive as spam or not spam.

1. We take all the words in a normal (not scam) email and create a plot (e.g. a histogram) to see how many times each word occurs in the normal email.
2. We can use the histogram (word count) to calculate the probabilities of seeing each word, given that it was a normal email.
   a. For example, the probability of a word “free” in a normal message = P(free|normal) = the total number of times “free” occurred in normal email (say 5) / the total number of words in a normal email (say 100).
      i. P(free|normal) = 5/100 = 0.05
   b. Similarly, the probability of a word “offer” in a normal message = P(offer|normal) = the total number of times “offer” occurred in normal email (say 8) / the total number of words in a normal email (say 100).
      i. P(offer|normal) = 8/100 = 0.08

Example of Multinomial Naive Bayes Classifier

3. Similarly, We can use the histogram (word count) to calculate the probabilities of seeing each word, given that it was a spam email.
   a. For example, the probability of a word “free” in a spam message = P(free|spam) = the total number of times “free” occurred in spam email (say 20) / the total number of words in a spam email (say 50).
      i. P(free|spam) = 20/50 = 0.4
   b. Similarly, the probability of a word “offer” in a spam message = P(offer|spam) = the total number of times “offer” occurred in spam email (say 40) / the total number of words in a spam email (say 50).
      i. P(offer|spam) = 40/50 = 0.8

Example of Multinomial Naive Bayes Classifier

4. Now, imagine you got a new email that said "*offer!*", and we want to decide if this is a normal email or spam.
5. We start with an initial guess about the probability that any email, regardless of what it says, is a normal email.
   a. We would estimate this guess from the training data.
   b. For example, since 100 of the 150 emails are normal, P(N) = 100/100+50 = 0.66
6. This initial guess that we observe a normal email is called "prior probability"
7. Now we multiply that initial guess by the probability that the word "*offer*" occurs in a normal email.
   a. P(N) x P(offer|normal) = 0.66 x 0.08 = 0.052
   b. We can think of 0.052 as the score that "*offer*" gets if it belongs to the normal email class.

Example of Multinomial Naive Bayes Classifier

9. Just like before, we start with an initial guess about the probability that any email, regardless of what it says is spam
   a. We would estimate this guess from the training data.
   b. For example, since 50 of the 150 emails are normal, P(S) = 50/100+50 = 0.33
10. Now we multiply that initial guess by the probability that the word "*offer*" occurs in a spam email.
   a. P(S) x P(offer|spam) = 0.33 x 0.8 = 0.26
   b. We can think of 0.26 as the score that "*offer*" gets if it belongs to the spam class.
11. Because the score we got for "*offer*" if it belongs to spam is > the score if it belongs to normal email, we will decide that "*offer!*" is a spam email. (0.26>0.052)

Example of Multinomial Naive Bayes Classifier

- What happens when we have to predict whether an email will go to spam or normal class with a message like "*free offer click offer offer*", where the new word "*click*" is not in the training dataset? P(click|normal) = 0 , P(click|spam) = 0.07
- If an email contains the word "*click*", it will never be classified as normal. That is a problem.
- To address this problem, we add 1 count to each word in the training data.
- The number of counts we add to each word is referred as α (alpha). You can set α to anything.
  - Now when we calculate probabilities of observing each word, we never get 0.
  - Adding counts to each word does not change the initial probability of an email is normal P(N) or spam P(S). Because adding a new word to each email did not change the number of emails that are already normal or spam.

So How is Naive Bayes "*Naive*"?

- Treats all word orders the same.
- For example, the normal email score for "*free offer*" is the exact same for "*offer free*"
  - P(N) x P(free|normal) x P(offer|normal) = P(N) x P(offer|normal) x P(free|normal)
- Treating all word orders the same is different from real world use of language.
- Naive Bayes ignores all types of grammar and language rules.
- By ignoring relationship between words, Naive Bayes has high bias, but because it works well in practice it has low variance.